

MRPL Sovereign Workbench — Demo-Day Risk List (Plain English)

This is every issue found so far — from the first code review, from checking against the official SIH problem statement (PS 26117), and from reviewing your team's "corrected workflow" plan — put in one list, ordered by **how likely it is to hurt you while you're actually running the demo in front of judges**. Each item says: what's wrong, why it matters on demo day, and how to fix it.

If you only have time to fix a handful of things, fix everything under  first, then .

CRITICAL — fix these before you rehearse

1. There's no real proof the system is offline — just a label

What's wrong: The screen shows a fixed green message saying "offline mode, no external connections." It doesn't actually check anything — it's just text that's always there.

Why it hurts: The problem statement says, almost word for word, that a claim of "sovereign/offline" is not enough — they want to *see* proof, through logs or a live monitor. This is one of only four things the judges are explicitly told to look for. If it's still just a label on demo day, you literally cannot perform this required part of the demo.

Fix: Add something that watches the system's actual network activity while it works and shows it live — e.g. a small counter on screen that ticks "0 external connections" in real time as you use the app, backed by something actually checking the network, not just a sentence that's always shown. Your team's plan says this is being built. Confirm it's actually working, don't assume the plan document means it's done.

2. Reading a scanned report and drafting a Word note needs to work in ONE message

What's wrong: In the original code, the system could either read an uploaded scanned document, or generate a Word file — but not both in the same request. It decides which tools it's "allowed" to use based on the wording of your message, and no wording gives it both at once.

Why it hurts: "Scan a report, pull out the findings, draft a Word approval note" is one of the four demo scenarios the problem statement explicitly asks for, almost word for word. If this still only works as two separate messages, it'll look broken — especially if a judge asks you to do it live with a document you haven't rehearsed with.

Fix: Your team's new plan does describe a fix for this (a dedicated pipeline that always has both the "read documents" tool and the "make a Word file" tool ready together for this exact request). Good — but **test it for real, multiple times, with a few different scanned documents**, before demo day. A plan describing a fix is not the same as the fix working.

3. Can one employee read another employee's private chat history?

What's wrong: In the original code, there was no separation between users. Anyone who opens the app sees a list of *everyone's* past conversations and can click into any of them.

Why it hurts: Your whole pitch is that this tool is trustworthy with confidential refinery data. If a judge opens the sidebar and sees they can read someone else's chat, that directly contradicts your pitch, live, in seconds.

Fix: Each employee's chat list should only show their own conversations — this needs a small change so chats are tagged with who they belong to and filtered accordingly. **Your team's corrected plan does not clearly say this got fixed** — it describes fixing something else (stopping one person's "approved" click from leaking into someone else's request, which is a different bug). Get a clear yes/no answer on this specific point before demo day. If the answer is no, either fix it, or don't demonstrate two "employees" side by side.

4. The system can misread your question and try the wrong tool

What's wrong: The system decides what you want by looking for certain words in your message. If your sentence happens to contain the word "document" (a completely normal word to use when asking about a scanned report), it can wrongly conclude you want it to *create* a Word file, and then it tries to open your scanned PDF as if it were already a Word file — which fails with an error.

Why it hurts: This might work perfectly on your rehearsed script and then break the moment a judge asks the same thing in slightly different words — which is exactly what happens in real Q&A.

Fix: The system should check the actual type of file that was uploaded *first*, and only guess from wording when there's no file attached. Test your planned demo question using 4–5 different phrasings, not just the one you rehearsed, to see if any of them break it.

● **HIGH RISK — probably fine on your rehearsed script, but risky if anything goes off-script**

5. The "which tool should I use" step isn't 100% reliable

What's wrong: A small AI model decides which tool to use next, and it's supposed to reply in a strict, computer-readable format. But nothing *forces* it to — it's just asked nicely to do so. Small AI models don't always follow that instruction perfectly.

Why it hurts: On your exact rehearsed wording it'll likely work every time. Ask something slightly different live, and there's a real chance this step silently fails or picks the wrong tool.

Fix: There's a setting that forces the AI to always answer in the correct format instead of just hoping it does. This is a small, one-line code change with a big reliability payoff — worth doing regardless of anything else on this list.

6. Long scanned documents get silently cut off after 5 pages

What's wrong: If a scanned document is more than 5 pages, the system only reads the first 5 and quietly stops — no warning, no error. It just answers based on incomplete information as if it read the whole thing.

Why it hurts: Judges sometimes bring their own test document instead of using yours. If theirs is longer than 5 pages, you'll get a confidently incomplete answer with no indication anything was skipped.

Fix (short-term, before demo): Make sure whatever document you plan to demo with is 5 pages or fewer. Also add a simple on-screen warning when a document goes over the limit, so at least it's honest. Properly removing the limit is more work and can wait until after the hackathon.

7. Excel and PowerPoint files skip the "ask for approval" step — only Word files get it

What's wrong: The system is meant to ask a human to click "approve" before doing anything sensitive. Right now that rule only covers making Word documents, running code, and sending email. Making an Excel or PowerPoint file happens with no approval step at all, for no stated reason.

Why it hurts: If a judge asks to see how the approval system works, and you then generate an Excel file with no approval prompt, it undercuts your "a human stays in control" pitch. Your team's corrected plan still lists the exact same three tools — this wasn't changed.

Fix: Either add Excel/PowerPoint generation to the approval-required list, or explicitly decide and explain why Word needs it and the others don't (e.g. "Word is used for official approval notes, spreadsheets aren't sensitive the same way"). Either is fine — just don't leave it as an unexplained gap.

8. The app restarts its tools from scratch on every single message

What's wrong: Every time you send a message that needs a tool (reading a file, making a document, etc.), the system shuts down and restarts its internal tool programs completely, every single time, instead of keeping them running between messages.

Why it hurts: On the kind of modest laptop this is designed to run on, this adds real, noticeable delay to every response, which looks sluggish in front of judges — especially compared to the cloud AI tools they use daily.

Fix: Keep those tool programs running between messages instead of restarting them each time. This takes more rework than the other items, but it's the difference between the demo feeling snappy vs. feeling laggy — worth doing if you have implementation time left.

9. Searching your SOPs/manuals matches words, not meaning

What's wrong: When the system searches your uploaded manuals, it isn't really "understanding" the text — it's doing a rough word-matching trick, chosen specifically to avoid downloading a proper AI search model.

Why it hurts: If a judge asks a question using different words than the manual itself uses (e.g. asking about "bearing wear" when the document says "vibration trending upward"), the search can miss the right passage entirely and the answer will look wrong or made-up — right on the exact feature (grounding answers in your own documents) the problem statement cares about most.

Fix: Swap in a real, small, fully-offline AI search model instead of the word-matching trick. More work than the other items, but it directly affects how good your answers look under live, unscripted questions.

● **MODERATE — won't break the demo, but could hurt if someone looks closely**

10. The code-execution safety check has a small gap

What's wrong: The system blocks a list of risky things a user's code could try to do, but it's missing one item from that list that could do something similar to what's already blocked.

Why it hurts: This is boxed in by other protections (no internet access, thrown away after each run), so it's very unlikely to cause a live failure — but a technical judge reading your code closely could point it out.

Fix: Add the missing item to the blocked list, or better, switch from "block these specific risky things" to "only allow this specific safe list of things."

11. The project's dependency list claims things that aren't actually used

What's wrong: The file listing what software the project needs says it uses two well-known AI frameworks — but the actual code never uses either of them. It also lists a search-quality tool that isn't used.

Why it hurts: No live-demo impact, but if a judge opens your code (some SIH rounds do review code, not just watch the demo), this kind of mismatch quietly makes your other claims look less trustworthy.

Fix: 20-minute cleanup — remove the unused entries, or actually start using them if time allows.

12. A settings file exists that the program never actually reads

What's wrong: There's a config file meant to control which tools get loaded, but the program ignores it and uses a different, hardcoded method instead.

Why it hurts: Same as above — only matters if someone reads the code closely.

Fix: Either wire it in for real, or delete it so it's not sitting there unused and confusing.

13. No automated tests, no one-command setup, no license file

What's wrong: Standard project basics are missing — nothing automatically checks the code still works, there's no single command to start everything, and there's no license file.

Why it hurts: Doesn't affect the live demo. Affects how "production-ready" the project looks if judges look past the demo and into the repository.

Fix: Add a few basic tests, a one-command setup, and a license. Lowest priority on this list — do it last, if time allows.

○ NICE TO HAVE — real improvements, not required for the demo

14. Can't yet answer questions using your sensor/incident/maintenance spreadsheets

What's wrong: You have real sample data (sensor readings, past incidents, maintenance logs) sitting in the project, but nothing in the code can read or use it yet.

Why it matters: Not one of the four things the problem statement requires you to demo, so it's not urgent — but it would be an impressive bonus if you have spare time, since the data is already there.

Fix: Build one more tool that can answer questions from those spreadsheets. Only after everything above is solid.

15. Some old sample files aren't connected to anything anymore

What's wrong: Leftover files from an earlier version of the project that the current code doesn't use.

Why it matters: Pure tidiness. Zero demo impact.

Fix: Delete them, or wire them in if there's a reason to keep them.

16. The "AI gets smarter over time" feature in your plan isn't built yet

What's wrong: Your corrected-workflow document describes a future feature where the system learns which AI model works best for which task over time. It's clearly and honestly labeled "not yet built."

Why it matters: Not required. I'd actually trim how much space this gets in the plan document right now, and put that effort toward double-checking items 1-4 above are real and working.

Fix: Leave it as a one-line "future work" note for now. Don't build it before the essentials are solid.

17. One claim in your plan needs double-checking: "the memory feature is built but never turned on"

What's wrong: Your team's corrected-workflow document says this, but it wasn't something I personally verified in the original code — I can't confirm it's true or false.

Why it matters: Only matters if you plan to demo "look, it remembers what I told it earlier." If that's part of your script, test that specific thing yourself first.

Fix: Have your friend's Claude session — the one with the real code — check this specifically and confirm either way before you rely on it.

Before demo day: a simple checklist

- ☐ Network/offline proof is real and visible on screen (#1)
- ☐ Scanned-report-to-Word-note works in one message, tested with 2-3 different documents (#2)
- ☐ Confirmed — not assumed — that one employee can't see another's chat (#3)
- ☐ Tested your demo question with different wording, not just the rehearsed line (#4)
- ☐ JSON format forced on the tool-picking step (#5)
- ☐ Demo document is under 5 pages, or a warning shows when it isn't (#6)
- ☐ Approval-step story is consistent across Word/Excel/PowerPoint, one way or the other (#7)